

FOX_Checklist

What this is: every FoxByte feature and its sub-features — built or not, and how it was tested — so it's clear where we are and what's next. How each feature works is in `FOX_Feature_Implemented.pdf` (same ids).

Last updated: 22 September 2026 · **Reflects:** `main @ 2778762` (v1.0.1, exact repository path PR #1) + the FoxByte rename and brand indirection (`chore/foxbyte`)

Source: `docs/FOX_Checklist.html` → `make feature-doc` → this PDF

✔ Done & tested — automated tests (unit, VM integration or CI)

🔸 Partial — gaps, or not verified end to end

Sub-features:

✔ built

🔸 partly

❑ not built yet

✖ broken

 · "VM §n" = section of `make integration` ; "v2 §n" = `make integration-v2` ; "update" = `make integration-update`

✔ Done — tested by hand only

❌ Known issue

❑ Not started

Where we are

Area	Features	Done & tested	Done (manual)	Partial	Known issue	Not started	Progress
A · Install & setup	9	4	3	2	·	·	78%
B · CLI & services	3	3	·	·	·	·	100%
C · Branching & storage	8	7	·	1	·	·	88%
D · Durability	6	5	·	1	·	·	83%
E · Security foundations	3	3	·	·	·	·	100%
F · SQL gateway	7	6	1	·	·	·	100%
G · Accounts & auth	6	5	·	1	·	·	83%
H · Blackbox (record of schema changes)	18	17	·	1	·	·	94%
I · Import & migration	13	5	8	·	·	·	100%
J · ETL pipelines (experimental)	4	2	2	·	·	·	100%
K · APIs, SDKs & agents	6	4	2	·	·	·	100%
L · Web console	11	1	10	·	·	·	100%
M · Build, CI & releases	7	6	1	·	·	·	100%
Total (features built or started)	101	68	27	6	·	·	94%
N · Not started (roadmap)	10	1	·	·	·	9	10%

Storage engine — not implemented, by design

FoxByte does **not** have its own storage engine. Data is stored by **PostgreSQL's own engine** (heap tables, B-tree indexes, write-ahead log) running on local disk inside the stock `postgres:18-bookworm` image plus `wal-g` (`postgres:16-bookworm` for an install created before 21 Sep 2026). This was a locked architecture decision: a Neon-style engine that separates compute from storage over the network adds latency to every transaction, and transaction speed must never be compromised.

What **is** built is a storage *layer* around Postgres: copy-on-write branch drivers — ZFS on macOS/Linux, btrfs on Windows — behind one interface in `internal/branch/storage.go` (C1), quotas (C6), capacity reporting (C7), and durability to object storage with `wal-g` + MinIO: WAL archiving, base backups, point-in-time restore (D1–D3). Networked storage disaggregation is listed in the README roadmap but not started (see "Not started").

Suggested next steps

- The first run and the record are now the built parts** (B3, L5, H13): sample data, a console that opens itself with the setup token in the link, `fox check`, `fox connect`, plain-English errors — and a Blackbox page that keeps itself up to date, shows each entry's place in the hash chain, and offers to branch from before a change on the row where the change is. Rewind is documented now too; before this it worked and appeared in no user-facing page.

2. Run on real machines what the throwaway VM cannot: `fox update` across a real release on macOS and Windows (A9); on Windows, the `zstd` distro image's unpack (M5), the `btrfs` holding area of `fox pg upgrade` (D6), and an export's copy to the host (D5).
3. Automated tests for the imports (I1–I3, I7) — the last area with none.
4. Gate release tags on the nightly suites, so a tag cannot ship past a red suite (M2).
5. Decide how this checklist shows partial work (audit v2 G31): 9 features are badged done while carrying an open sub-item (C7, D5, D6, F5, G3, H16, H17, I13, M7) — down from 34. Each of those sub-items is a disclosed limit rather than unfinished work, so the remaining question is whether a limit belongs in the sub-item list at all.
6. **From the audit, deliberately left:** HA beyond the demo — automatic failover and fencing across hosts (D4, G21); metrics, alerts and restart on crash (B2, G27); branch merge (H18, G29); timeouts on the engine's own commands and splitting the branch package (G30); quotas on `btrfs` and per-branch CPU/memory (C6, G33); `pgvector` in the image (M6, G34); `/api/v1` (K1, G35).

ID	Feature · sub-features	Tested by
 A1	One-line installer (macOS / Linux) ✓ Detects OS / CPU, downloads the release binary ✓ Installs to the prefix (sudo only if needed) ✓ Stages the Linux engine for the VM on macOS	Unit (<code>test_install_sh.sh</code> , CI); manual
 A2	Windows installer (install.ps1) ✓ Installs WSL, elevates, resumes after reboot ✓ Downloads fox.exe, engine, image build context ✓ Ubuntu rootfs verified against Ubuntu's checksums ✓ Adds to PATH and runs <code>fox setup</code>	Pester (Windows CI); Windows run 16 Sep
 A3	fox setup on macOS (Lima) ✓ Creates / starts the Lima VM ✓ Installs Docker and ZFS ✓ Reinstalls the engine on every run ✓ Starts the stack	Unit; run against a deliberately broken VM, 21 Sep
 A4	fox setup on Windows (WSL2 + btrfs) ✓ Imports a dedicated distro with systemd ✓ Docker, btrfs and a storage mount unit ✓ Quiet steps, detail in install.log ✓ Waits for systemd on a cold distro	Unit (<code>host_wsl_test.go</code>); Windows run 16 Sep
 A5	Linux native ✓ Engine runs in-process, no VM ✓ <code>fox setup</code> explains nothing is needed	VM integration (all suites run on Linux)
 A6	Command forwarding & host file streaming ✓ macOS → Lima VM, starts the VM if stopped ✓ Windows → WSL distro	Unit (<code>TestImportLocalFile</code> , <code>TestWSLargs</code>)
 A7	Always-latest engine binary ✓ Downloads the newest engine into the cache (ELF check) ✓ <code>FOX_NO_REFRESH</code> , <code>FOX_VERSION</code>	Unit (<code>host_test.go</code> , <code>update_test.go</code>); Windows run 16 Sep
 A8	New-release notice on fox start ✓ Background check, 1.5 s limit, printed last ✓ Only a promoted, newer, complete release	Unit; update §1, §1b, §2, §3, §8; real Mac check
 A9	fox update ✓ Downloads and verifies everything before changing anything ✓ Stops servers, installs engine, restarts old one on failure ✓ <code>up</code> , Blackbox upgrade, <code>start</code> on the new engine ✓ Replaces host <code>fox</code> , keeps previous binaries	Unit; update §4–§7; a real Mac run

ID	Feature · sub-features	Tested by
----	------------------------	-----------

✓ B1	The fox command set ✓ Stack: setup, start, stop, up, down, status, check, logs, connect, psql, demo, update ✓ Branching, backup / restore, HA ✓ Blackbox, policy, impact ✓ Import, pipelines ✓ backup export / restore, pg status / upgrade	✓ Users, API keys, admin ✓ <code>fox vm status / fox vm shell</code> (Lima, WSL; Linux says there is no VM) ✓ <code>branch create --from</code> ✓ <code>fox uninstall</code> removes it all, retired installs included	Unit (<code>main_test.go</code> , <code>update_test.go</code>); VM suites
✓ B2	Background services (daemon) ✓ Control plane :8080, gateway :6432, agent API :8088 ✓ Detached, pidfiles and logs in <code>~/ .fox</code>	✓ Stop (TERM then KILL), status, logs □ Health check / restart on crash	Unit (<code>TestStopTerminatesProcess</code>); VM §1
✓ B3	First run (sample data, check, connect) ✓ Sample data seeded into <code>main</code> on a first start: <code>users</code> , <code>projects</code> , <code>events</code> , five rows each, two foreign keys ✓ <code>fox demo seed drop sql</code> on any branch; <code>FOX_NO_DEMO=1</code> starts empty ✓ The console opens itself, with the one-time setup token already in the link ✓ Declines to open a browser in the VM, over SSH, with no display, or with <code>FOX_NO_BROWSER</code>	✓ <code>fox check</code> : version, console, storage, main, three servers, three ports, TLS, Blackbox, backup age, sample data — each failure with the command that fixes it ✓ <code>fox connect [branch]</code> prints the application's connection string, then opens psql; <code>fox psql</code> takes a branch ✓ <code>fox branch create</code> ends with the connection string and how long it took ✓ Plain-English errors in the CLI and the console (<code>internal/errhint</code> , <code>web/src/errhint.ts</code>)	Unit; v1 §11d

C Branching & storage

7 of 8 done

ID	Feature · sub-features	Tested by
✓ C1	Storage drivers (ZFS / btrfs) ✓ ZFS snapshot + clone per branch (macOS VM, Linux) ✓ btrfs subvolume snapshots (Windows) ✓ Pool / filesystem created on a loopback file	Unit; VM §2, §2b (ZFS); btrfs manual on Windows
✓ C2	Branch lifecycle ✓ Create, list, delete ✓ Reset to parent ✓ Suspend / resume ✓ Recreate a container from its surviving data	Unit; VM §2, §2b, §4
✓ C3	Protection of main ✓ Delete and reset refused	Unit; VM §4, §6
✓ C4	Private networking (no host ports) ✓ Branch databases reachable only through the gateway ✓ Published ports (object store, restore, debug) on loopback only; standby unpublished	Unit (<code>TestParsePublishedPort</code> , <code>TestDSN</code>); VM §1c
✓ C5	Scale-to-zero & wake on connect ✓ Idle branches suspended automatically ✓ Next connection wakes the branch (~0.3–0.7 s)	VM §7
🔥 C6	Storage quotas ✓ Space reserved for main (ZFS) 🔥 Per-branch cap (opt-in)	Manual
✓ C7	Capacity & status ✓ Services, main health, backups, branches ✓ Storage used / available	VM §1
✓ C8	Provisioning & image pull ✓ Docker, storage pool and image prepared on up	Unit; VM §11b

ID	Feature · sub-features	Tested by
✓ D1	WAL archiving to object storage ✓ wal-g ships main's WAL to MinIO, off the commit path ✓ MinIO images pinned on quay.io ✓ Branches are disposable by design (export one that matters)	VM §3; VM §11c
✓ D2	Base backups ✓ Create and list ✓ Follow the primary after a failover; listing works with main stopped ✓ Read-only listing: <code>GET /api/backups</code> and a Dashboard panel	VM §3, §3b, §6; VM §11c
✓ D3	Point-in-time restore ✓ Restore to a timestamp or latest on :5433 ✓ Per-install credentials ✓ Starts from the newest base backup that precedes the target	Unit; VM §3, §3b, §6; v2 §1a
🔥 D4	HA standby & failover ✓ Streaming hot standby ✓ Manual failover, <code>main</code> rerouted ✓ Nothing deletes or stops the promoted standby ✓ <code>fox up</code> brings it back, not the old main	Unit; VM §6, §7
✓ D5	Portable export & restore ✓ Every non-agent branch: <code>pg_dump</code>, its roles, the Blackbox head; SHA256SUMS over all ✓ Copied out of the VM and verified on the host; a copy that fails is deleted ✓ Damaged, incomplete or path-escaping archives refused	Unit; VM pg-upgrade §1
✓ D6	PostgreSQL major upgrade ✓ <code>fox pg status</code>: the major of the install and of each branch ✓ Plan with every change crossed, found-in-your-data first; refusals say what to do ✓ Asks, exports, then reloads every carried branch on the new major	Unit; VM pg-upgrade (72 / 72)

E Security foundations

ID	Feature · sub-features	Tested by
✓ E1	Per-install generated secrets ✓ Generated Postgres and MinIO passwords ✓ Used by restore too ✓ Each login role has its own derived password	Unit (indirect); v2 §1a; VM §1d
✓ E2	TLS everywhere ✓ Self-signed TLS on all three services, zero setup	Unit (<code>TestReadStartupUpgradesTLS</code>)

✓	E3 Keys are made by a person, not by start	Integration §1
✓	Start creates no account and no key, and caches none on disk	✓ First account over HTTP needs the setup token (fox setup-token)
✓	The banner shows the connection string and how to make a key	✓ Exactly one first account, however many sign up at once
✓	The first account created gets the admin grant on main	

F SQL gateway

7 of 7 done

ID	Feature · sub-features	Tested by
✓	F1 Routing by database name	Unit; VM §1, §6
✓	One endpoint :6432, database name = branch	□ Connection pooling, rate limiting
✓	F2 API-key authentication	VM §1
✓	key_ key as the Postgres password	✓ Startup deadline and connection cap
✓	Bad key rejected with a Postgres error	✓ An account key opens main, its own branches, or any for an admin
✓	F3 Backend login (SCRAM)	VM §1, §6, §8 (exercised)
✓	Cleartext, md5, SCRAM-SHA-256	□ Unit test for SCRAM
✓	Clients never see the database password	
✓	F4 Per-user Postgres login roles	Manual
✓	Role per user, actor can't be forged	✓ Own password per role
✓	Permissions of non-superuser db_client	
✓	F5 Attribution session settings	VM §8
✓	Actor kind, branch, session id per connection	○ Kind / branch / session client-settable
✓	F6 Ctrl-C (CancelRequest) forwarding	Unit
✓	Cancel a running query through the gateway	
✓	F7 Errors & insecure build tag	Unit (TestSendError)
✓	Real Postgres error responses	✓ No-auth bypass impossible in release builds

G Accounts & auth

5 of 6 done

ID	Feature · sub-features	Tested by
✓	G1 Auth store (SQLite)	Unit (TestVerifyKeyConcurrent , TestConcurrentFirstOpen); VM §1
✓	Users, keys, sessions, OAuth ids, pipelines	□ Roles / permissions, multi-tenant projects
✓	Safe under concurrent use (WAL, busy timeout), including a new store created by three services at once	
✓	G2 Email & password users	Unit; VM §1; VM §1d
✓	Create account, log in	✓ List and delete users (admins)
✓	Password change (signs out other sessions); admin reset at the CLI	✓ Emails validated; at most 63 bytes (a role name)
✓	Self-service reset by email	
✓	G3 API keys	Unit; VM §1
✓	Create (shown once), list, revoke	○ Revoking an unknown id reports success
✓	Bearer / X-API-Key and gateway password	
✓	G4 Web sessions & signup control	Unit (TestSessions)
✓	httpOnly session cookie	✓ Failed sign-ins throttled
✓	Sign-up closed by default (FOX_SIGNUP=open)	

	G5 OAuth (GitHub / Google)	✓ Sign-in flows built ✓ Defaults match what the engine serves (<code>https://localhost:8080</code> ; web origin = public URL) ✓ Session cookie <code>Secure</code> / <code>SameSite</code> follow the origin	☐ Tested with a real OAuth app ✓ Only a provider-verified email joins or creates an account; closed sign-up respected ✓ State cookie one-use, <code>Secure</code> , <code>SameSite=Lax</code>	Unit; VM §1b
---	-----------------------------------	--	--	--------------

	G6 Security log	✓ Sign-ins, failures, keys, passwords, accounts, admin grants, refusals ✓ Hash-chained, append-only, anchored and signed	✓ <code>fox audit</code>, <code>verify</code>, <code>GET /api/audit</code> ☐ Included in <code>fox backup export</code>	Unit; VM §1d
---	------------------------	---	---	--------------

H Blackbox (record of schema changes)

17 of 18 done

ID	Feature · sub-features	Tested by
	H1 DDL recording ✓ Who, tool, branch, statement, status, risk ✓ Inherited by every branch ✓ A restart records nothing: the install script's own statements are not captured ☐ Role / database changes, some dropped object types	VM §8; v2 §9; v2 §10
	H2 Hash chain, append-only & verify ✓ Rows can't be edited or deleted normally ✓ <code>fox ledger verify</code>  Superuser rewrite caught only via anchors (H10–H12)	VM §8; v2 §0, §6b
	H3 Attribution ✓ Humans via their login role ✓ Agents via their own <code>agent-<id></code> role	Unit; VM §8; v2 §1c
	H4 Destructive-DDL guardrail ✓ DROP TABLE / DROP SCHEMA refused, attempt recorded ✓ Override only superusers / <code>db_admin</code> , use recorded	VM §8; v2 §1b, §6b
	H5 Risk flags ✓ Type change, drop column flagged ✓ Judged on the running statement (no comments or literals)	Unit; v2 §6b
	H6 Blackbox access (CLI / REST / MCP / UI / SDK) ✓ CLI, REST with filters, MCP, web page, SDKs	v2 §0, §5
	H7 <code>fox ledger revert --to</code> ✓ Help and output say it's a point-in-time restore of main	Unit (help text); VM §3 (the restore)
	H8 Override permission (<code>db_admin</code>) & upgrade ✓ <code>fox admin grant revoke list</code> ✓ First user granted automatically	v2 §1b, §6
	H9 Blackbox 2.0 capture ✓ Transaction id and WAL position per entry ✓ Task, tool-call hash, override use	Unit; v2 §2
	H10 Checkpoints & anchors ✓ Merkle root in the database and in read-only anchor files ✓ On a schedule and on demand ✓ Published anchor format	Unit; v2 §3, §3b

✓	H11 fox-verify (independent verifier)	✓ Checks a live branch or an export ✓ Release binaries for every platform	✓ <code>--pubkey</code> checks signatures	Unit; v2 §3
✓	H12 Blackbox integrity & export	✓ INTACT / TAMPERED via CLI, REST, MCP ✓ JSON-lines export	□ Streaming for very large ledgers	v2 §3
✓	H13 Branch from just before a Blackbox entry	✓ Restore + replay up to the change's transaction ✓ By transaction id or timestamp ✓ CLI, REST, MCP, the Integrity panel, and every row of the Blackbox page	✓ main untouched ✓ Documented: README, the Guide's step 6, and the reference	Unit; v2 §4; VM ui
✓	H14 Blackbox names & aliases	✓ <code>fox_blackbox</code>, <code>/blackbox</code> routes, MCP and SDK names	✓ Every original name still works	Unit; v2 §5
✓	H15 Policy gate (warn / block)	✓ BBX02 warn / BBX01 block with JSON detail ✓ Blocked attempts recorded ✓ Per-rule <code>db_admin</code> override	✓ CLI, REST, MCP, web page; custom rules ✓ Patterns match the running statement ✓ TRUNCATE (its own guardrail, H4)	Unit; v2 §6, §6b
✓	H16 Agent provenance & execute_change	✓ Session, task, parent session on every agent change (also through the Gateway DSN) ✓ MCP <code>execute_change</code> with policy preview	✓ <code>fox_blackbox_sessions</code>, REST 🟡 Provenance is what the agent says (not verified)	Unit; v2 §7
✓	H17 Impact analysis	✓ Dependents (views, foreign keys, indexes, depth) ✓ Other branches that have the object ✓ Policy verdict and risk score	✓ CLI, REST, MCP, inside <code>execute_change</code> 🟡 Functions, types, schemas not recognised from SQL	Unit; v2 §8
✓	H18 Blackbox diff, and promoting a branch's changes	✓ Fork point, changes only on each side ✓ Objects changed on both ✓ CLI, REST, MCP ✓ Promotion: a branch's recorded statements, reviewed, then applied to another branch in one transaction	✓ Refused on a conflict (both sides changed the same object), on a statement the target's gate blocks, and when there is nothing to promote ✓ Approving needs the right to manage the target, so an agent cannot approve its own request ✓ CLI, REST, MCP and the web Requests page	Unit; v2 §8, §8b

I Import & migration

13 of 13 done

ID	Feature · sub-features	Tested by
✓ I1	From Postgres ✓ Schema and data via <code>pg_dump</code> ✓ Zero-table result reported as an error	Manual
✓ I2	From MySQL ≤5.7 / MariaDB ✓ pgloader with type mapping (multi-arch image)	Manual
✓ I3	From MySQL 8.x ✓ Native mysqldump path (<code>caching_sha2_auth</code>) ✓ Types, NULLs, unicode, PK	Manual
✓ I4	From MongoDB ✓ Collection → typed table, exact name	VM §9, §10

✓	I5	From a .sql dump	✓ Runs the dump into a new branch ✓ Fails listing every failed statement (instance kept)	✓ Same for Postgres imports (I1) ✓ Automated test	Unit; VM §11
✓	I6	From JSON / NDJSON	✓ Columns inferred from top-level keys	✓ An invalid document fails the import, naming its element or line	Unit; VM §10, §11
✓	I7	From CSV	✓ Table from the header, exact column names	□ Typed columns (all text today)	Manual
✓	I8	Continuous replication & cutover	✓ Initial copy + streaming (logical replication) ✓ Cutover keeps data (CLI) ✓ Status and cutover from REST and the Import page	✓ A replicating branch is never auto-suspended ✓ Automated test (VM §5b) and driven in the browser	Unit; VM §5b
✓	I9	Guard handling during import	✓ Guardrail paused, recording stays on	✓ Re-armed even after a crash	Manual
✓	I10	Live progress (CLI & SSE)	✓ Live log and progress bar	🟡 psql subprocess output not streamed	Manual
✓	I11	Import REST endpoints	✓ From a connection string or an uploaded file □ Target name validation	✓ Private, local and container sources for admins only; metadata refused	Manual; unit; VM §1d
✓	I12	Web Import page	✓ Database and file panels with live console	✓ No longer claims a SQLite loader that doesn't exist	Manual
✓	I13	Exact identifier fidelity	✓ Case kept (<code>userId</code> , <code>leaseAiChats</code>)	🟡 Field order of JSON sources not kept	Unit; VM §10

J ETL pipelines (experimental)

4 of 4 done

ID	Feature · sub-features	Tested by
 J1	Pipeline engine ✓ Extract → raw schema → SQL models → tests, in a fresh branch ✓ <code>ref()</code> / <code>source()</code> templating ☐ Dependency graph ☐ Incremental models	Unit; VM §9
	☐ Scheduling ✓ A run replaces only a branch the pipeline made ✓ API sources are connection strings, never server paths	
 J2	Data-quality tests ✓ <code>not_null</code>, <code>unique</code>, <code>accepted_values</code>, <code>row_count_min</code>, <code>custom</code>	Unit; VM §9
	✓ Failure → exit 1, data kept	
 J3	Stored pipelines & run history ✓ Saved per user, run over the API, history ☐ Read a run's log	Manual
	☐ Mark runs whose process died	
 J4	Pipeline editor (web) ✓ Create, edit, run, results, history	Manual
	☐ Confirm before delete	

K APIs, SDKs & agents

6 of 6 done

ID	Feature · sub-features		Tested by
 K1	Control-plane REST API		Unit; VM §1; v2 §0–§8; VM §1d
	 Branches, query, Blackbox, policy, impact, import, pipelines, keys  Versioned paths (/api/v1)	 Exact-origin CORS, branch-name check, body limits, timeouts  Branches owned by their maker; others 404 (owner, admin, or main)	

✓	K2 OpenAPI spec	✓ Served, Blackbox 2.0 endpoints documented ✓ Matches the code (import fields, status codes, auth routes, /blackbox aliases) ✓ Drift check in CI	Unit (drift check)
✓	K3 Python SDK	✓ Branches, query, Blackbox □ Import, pipelines, keys, agents □ Tests □ Publish to PyPI ✓ Contract tests against a live engine	Manual; VM sdks
✓	K4 TypeScript SDK	✓ Same client for Node and browsers □ Tests □ Publish to npm ✓ Compiles, and contract tests against a live engine	Manual; VM sdks
✓	K5 Agent Branch API	✓ A branch per agent, own non-superuser role ✓ Provenance (task, parent, session) ✓ DSN reachable from the host (gateway-routed, branch-scoped key) ✓ Cap, listing and TTL count suspended branches ✓ Superuser switch only in <code>--tags insecure</code> builds ✓ Agents and their keys belong to the caller	Unit; VM §5; v2 §1c, §7
✓	K6 MCP server	✓ Branches, run_sql, Blackbox tools ✓ policy_check, execute_change, impact, diff ✓ Requires an API key and acts as that account; a scoped key is held to its branch ✓ Client config & 16-tool reference (docs/mcp.md) ✓ Same ownership rule as the REST API	Unit; v2 §1c–§8

L Web console

11 of 11 done

ID	Feature · sub-features	Tested by
✓ L1	Embedded single-origin UI ✓ Served by the engine at :8080 □ Automated UI tests (whole area)	Manual
✓ L2	Login & signup ✓ Log in / sign up tabs, OAuth buttons ✓ Covered by the console's Playwright tests	Manual; VM ui
✓ L3	Dashboard ✓ Live tiles ✓ Branch table: create, suspend, resume, delete, copy DSN ✓ Covered by the console's Playwright tests	Manual; VM ui
✓ L4	SQL console & browser ✓ Schema sidebar, rows, structure, indexes ✓ Values shown as Postgres writes them (uuid, interval, inet, bytea) ✓ SQL editor ✓ System tables hidden by default, with a show button ✓ Runs as the signed-in user; one-run override switch ✓ Per-rule override for policy blocks 🟡 One statement per run (extended protocol)	Manual; API in v2 §6
✓ L5	Blackbox page ✓ Entries per branch with badges ✓ Table / risk / date filters, and "Clear filters" ✓ Verify button; a row expands to its statement and session ✓ Live: new entries arrive on their own (3s), marked briefly, pausable; never while older pages are being read ✓ The chain is visible: entry id, its row hash and the previous entry's, each copyable ✓ "Branch from before this change" on the row itself (<code>main</code> ; explained elsewhere) ✓ Covered by the console's Playwright tests	v2 §5b; browser; VM ui
✓ L6	API keys page ✓ Create, copy, list, revoke ✓ A new key is shown inside its connection string (E3's next step) ✓ Covered by the console's Playwright tests	Manual; browser review 21 Sep; VM ui
✓ L7	Landing, Guide & Docs pages ✓ Landing, developer guide, reference ✓ Refreshed against the code (https, sslmode, MCP, Blackbox, import routes)	Type-check + build

✓	L8 Blackbox integrity page	✓ Verify, checkpoint, export, branch-before ✓ Used signed in; every result matched the CLI	🟡 Entry list needs a refresh to show a new change	Type-check + build; browser review 16 Sep
✓	L9 Policies page	✓ Rules, warn/block, custom rules, preview, evaluations ✓ Used signed in; matched the CLI, wrote only to the chosen branch	✓ Non-admin change refused with a readable message ✓ Who can override: list, grant, revoke	Type-check + build; API in v2 §6; browser review 16 Sep
✓	L10 App sidebar layout	✓ Every link opens its page, highlighted, with breadcrumb ✓ Collapse persists across a reload; theme toggle persists	✓ No console errors on any page ❑ Under-860px drawer (needs a manual look)	Type-check + build; browser review 16 Sep
✓	L11 Design system & FoxByte theme	✓ Tokens for colour, type, space, radius, elevation and motion; components use no raw values ✓ Ember palette in both themes; filled controls legible (<code>--accent-ink</code>) ✓ Fox mark from theme variables, wordmark from brand.json, favicon	✓ Console, Blackbox, Policies and API keys re-laid out; one table style ✓ Every page reviewed in both themes; console checked at 375px ❑ Not yet reviewed against a running engine with real data	Type-check + build; browser review 21 Sep

M

Build, CI & releases

7 of 7 done

ID	Feature · sub-features	Tested by
✓ M1	Makefile targets ✓ build, vet, test, integration suites ✓ Suites run only in a throwaway test VM (guarded)	CI ✓ release, web, WSL builds, feature-doc (both PDFs) ✓ <code>integration-sdks</code>, <code>integration-ui</code>
✓ M2	Linux CI ✓ Build, vet, test, gofmt ✓ Release script tests, checklist sync check ✓ VM integration suites nightly in CI (one runner per suite)	CI ❑ Release tags gated on them ✓ Secret scanning, govulncheck, npm audit, UI type-check
✓ M3	Windows CI ✓ Build, vet, test, Pester, installer encoding ✓ Guard tests read backslash paths, and generated files are pinned to LF (21 Sep)	CI ❑ End-to-end WSL install test
✓ M4	Releases & GHCR image ✓ Binaries for 5 platforms + fox-verify ✓ SHA256SUMS merged safely between jobs ✓ Fails unless <code>fox update</code>'s files are published	Script tests in CI ✓ Multi-arch Postgres image on GHCR ✓ Actions pinned by SHA; signed SHA256SUMS; build attestations; SBOM ❑ First signed release run on GitHub
✓ M5	WSL distro & ZFS builds ✓ Prebuilt distro image ✓ Downloaded and verified by the installer (seen on Windows) ✓ Preloaded image tagged as the engine runs it	Actions artifacts; Pester; Windows run 16 Sep ✓ Imported on a fresh Windows install; preloaded images loaded ✓ Published zstd-compressed, with the gzip size measured beside it in every build log 🟡 zstd unpack on a real Windows install (unit-tested only); the gzip image was 685 MB
✓ M6	Postgres image ✓ PostgreSQL 18 + wal-g for a fresh install; one image per supported major (16, 18) ✓ The image follows the data: an install on 16 keeps running 16 after an update ✓ A branch on the wrong major is refused before <code>docker run</code>, with the way out ✓ The major lives in one place, and a test holds the Dockerfile, release matrix and Windows preload to it	Unit; all three suites on 18 ✓ All three suites pass on PostgreSQL 18.6: 108 + 277 + 51 checks (21 Sep) ✓ A command that moves an install from 16 to 18 (D6), and a portable backup to take first (D5) ❑ pgvector ✓ Base image pinned by digest; wal-g checked by SHA-256

**M7****Product name in one file**

- ✓ brand.json → Go, shell, web and installer blocks, regenerated by `make brand`
- ✓ Stored names brand-free and frozen: a rename never migrates an install
- ✓ Retired env prefixes, state directory, VM names and installs still handled

- ✓ Guards: retired names anywhere, product names in stored SQL, durable names spelt twice
- The sweep is text substitution: review its diff (it over-reached twice)

Unit (brand, gen, oldnames), in CI via go test

Planned or discussed, nothing built yet. When work starts, move the row into its area with the next id there.

ID	Item · scope	Related
☐	Multi-tenant projects & roles ☐ Projects / organisations ☐ Per-branch permissions (RBAC)	G1, H6
☐	Scheduled backups & retention	D2
☐	Automatic HA failover ☐ Health detection, fencing ☐ Multi-host	D4
☐	Pipelines: scheduling, incremental / CDC extract, DAG view	J1
☐	More import sources: Cassandra, DynamoDB	I1–I8
☐	pgvector in the Postgres image	M6
☐	Cloud write-once anchors (S3 Object Lock etc.)	H10
☒	Release signatures — done 22 Sep 2026	A9, M4
☐	Hosted cloud tier	—
☐	Networked storage disaggregation (README roadmap; see the storage engine note on page 1)	C1, D1

Out of scope by decision

A custom storage engine (PostgreSQL's own engine is used, for native transaction speed), sharding, Raft / distributed transactions, multi-region, HTAP.